
Sparsity on VLMs - token pruning

YoungJo Choi
New York University
yc7330@nyu.edu

Julia Gontijo Lopes
New York University
jg19879@nyu.edu

Shubham Ojha
New York University
so2561@nyu.edu

1 Introduction

Vision-Language Models (VLMs) achieve impressive multimodal understanding by processing visual inputs as token sequences jointly attended with text in transformer decoders. However, this design introduces a scalability challenge: visual token count grows rapidly with image resolution, and transformer attention scales quadratically with sequence length $\mathcal{O}(n^2)$. For instance, LLaVA Liu et al. [2024] encodes a 336×336 image into 576 visual tokens, making visual attention the dominant inference bottleneck.

Traditional efficiency approaches focus on weight pruning - removing model parameters based on magnitude or structure. However, weight pruning does not reduce sequence length, providing minimal speedup without specialized sparse hardware. We hypothesize that *token pruning* is more effective for VLMs because it directly addresses the quadratic attention bottleneck.

In this work, we systematically compare weight pruning versus token pruning for VLM efficiency. We evaluate three weight pruning methods (unstructured, kernel, and filter) on LLaVA-1.5’s CLIP vision encoder and compare against SparseVLM Zhang et al. [2025], a text-guided visual token sparsification method.

Our experiments reveal three key findings:

1. **Weight pruning is ineffective.** Unstructured pruning preserves accuracy but provides $<3\%$ speedup. Structured pruning causes catastrophic failure—even at 5% sparsity—due to CLIP’s sensitivity to structural modifications.
2. **Token pruning addresses the bottleneck.** SparseVLM achieves 18–20% latency reduction with only 5.5% accuracy drop by removing redundant visual tokens.
3. **Combining strategies yields further gains.** Integrating SparseVLM’s text-guided selection with FastV’s shallow-layer pruning achieves superior efficiency-accuracy trade-offs.

Our results demonstrate that token reduction is fundamentally more effective than weight reduction for VLM efficiency, with important implications for designing efficient multimodal systems.

2 Individual Contribution

* YoungJo (Jo) Choi: Jo has analyzed visual token pruning; SparseVLM in particular, and compared it with other weight pruning techniques. Jo has shown experimentally that the advantages of visual token pruning over weight pruning.

* Julia Gontijo Lopes: studied where to prune visual tokens in VLM decoders, analyzing pruning at different layers. She implemented FastV-style early-layer pruning and combined it with SparseVLM’s text-guided token selection, achieving improved efficiency–accuracy trade-offs.

* Shubham Ojha: Shubham conducts a comparative analysis of vision token pruning strategies in VLMs, evaluating and contrasting broad classes of pruning methods and their design principles.

3 Problem Description

Vision–Language Models (VLMs) have advanced rapidly in their ability to jointly reason over visual and textual modalities, enabling applications such as high-resolution image understanding and long-video conversational analysis. Despite these successes, scalability remains a fundamental challenge. In this work, we show that the primary inference bottleneck stems from the rapid growth in the number of visual tokens, which increasingly dominate end-to-end latency. We argue that this bottleneck is more effectively addressed by optimizing visual tokenization and token selection strategies, rather than relying solely on pruning or compressing model weights. Furthermore, existing literature has devoted limited attention to systematic approaches for visual token pruning in VLMs. To bridge this gap, we conduct a comprehensive study of multiple visual token pruning and selection techniques, analyzing their impact on inference latency and model performance.

4 Related Work

4.1 Vision-Language Models

Recent advances in vision–language models have improved multimodal comprehension and generation by enabling the processing of longer visual token sequences. However, the use of higher-resolution images inevitably leads to a rapid—and often exponential—increase in the number of visual tokens. For instance, LLaVA encodes a 336×336 image into 576 visual tokens, while increasing the resolution to 672×672 results in up to 2,880 visual tokens. This substantial growth in visual sequence length introduces a significant computational bottleneck during inference. Consequently, further research on visual token sparsification is critical to fully unlock the scalability and efficiency of vision–language models.

4.2 Visual Compression for VLMs

Compressing visual tokens is essential for efficient vision–language modeling. First, the number of visual tokens is typically tens to hundreds of times larger than that of language tokens, leading to disproportionate computational costs. Second, visual representations are inherently more information-sparse than human-generated text, containing substantial redundancy across spatial and temporal dimensions.

Existing approaches to address this challenge can be broadly categorized into three directions:

- 1) Vision Token Pruning in Encoders (Text-Agnostic): Pruning visual tokens based solely on visual features, without relying on textual guidance.
- 2) Vision Token Pruning in Decoders (Text-Guided): Selecting or pruning visual tokens using textual context to retain only those most relevant to the language input.
- 3) Vision Token Pruning in Both Encoders and Decoders: Jointly applying pruning strategies across both stages to maximize efficiency while preserving multimodal performance.

5 Method

5.1 Weight Pruning Techniques

We evaluate three weight pruning methods applied to the CLIP vision encoder of LLaVA-1.5 (7B). All pruning is performed without fine-tuning to assess the raw impact on pretrained representations.

Unstructured Pruning removes individual weights based on their magnitude. Weights with absolute values below a threshold (determined by the target sparsity ratio) are set to zero. This method preserves network structure while introducing sparsity, but requires specialized hardware for inference speedup since standard GPUs cannot skip zero-valued computations.

Kernel Pruning is a structured method that removes entire 3×3 convolutional kernels based on their L2 norm. Each kernel acts as a feature detector (e.g., edges, textures), so removing kernels eliminates specific detection capabilities from the network.

Filter Pruning is the most aggressive structured method, removing entire output channels from convolutional layers. Each filter comprises all kernels producing one feature map, so filter removal reduces the output dimensionality and affects all downstream layers.

5.2 SparseVLM: Text-Guided Visual Token Sparsification

Unlike weight pruning methods that modify model parameters, SparseVLM Zhang et al. [2025] operates at the token level by selectively removing redundant visual tokens based on their relevance to the input question.

In LLaVA, a 336×336 image produces 576 visual tokens after CLIP encoding. Since transformer attention scales quadratically with sequence length $\mathcal{O}(n^2)$, this visual token count becomes a primary bottleneck for inference speed.

SparseVLM addresses this by computing cross-attention scores between text and visual tokens to determine visual token importance. Visual tokens that receive high attention from question-relevant text tokens are retained, while tokens representing irrelevant regions (e.g., background, sky) are discarded.

Given the computed importance scores, we select the top- k visual tokens with the highest scores, where k is determined by the target keep ratio. For example, with a 30% sparsity target (70% keep ratio), we retain 403 out of 576 tokens. The selected tokens maintain their original order to preserve spatial relationships in the image. The sparse visual representation is then projected and concatenated with text tokens as in the standard LLaVA pipeline, but with a reduced sequence length.

Key advantages over weight pruning include: (1) training-free deployment on any pretrained checkpoint, (2) direct reduction of the quadratic attention bottleneck, and (3) preservation of the complete model architecture.

We implement SparseVLM from scratch using HuggingFace Transformers components, as the official implementation requires Flash Attention 2, which was unavailable in our environment due to CUDA compatibility constraints. Our implementation follows the core methodology of Zhang et al. Zhang et al. [2025], utilizing text-guided cross-attention scoring for token selection.

5.3 Experimental Setup

Model. We use LLaVA-1.5 (7B), comprising a CLIP-ViT-L-336px vision encoder, a two-layer MLP projector, and Vicuna-7B. The vision encoder produces 576 visual tokens from 336×336 images.

Dataset. We evaluate on VQAv2-small, containing image-question pairs covering object recognition, counting, color identification, and spatial reasoning.

Pruning Implementation. We apply pruning exclusively to the CLIP vision encoder while keeping the LLM and projector intact. For unstructured pruning, weights below a magnitude threshold are zeroed. For kernel pruning, entire 3×3 kernels with low L2 norms are removed. For filter pruning, entire output channels with the lowest L2 norms are removed. We evaluate sparsity levels of 5%, 10%, 15%, 20%, 25%, and 30% without fine-tuning.

5.4 Evaluation Metrics

We evaluate performance using four metrics:

- **Containment Accuracy:** Whether the ground truth answer appears anywhere within the model’s response. This metric suits LLaVA’s verbose outputs (e.g., ground truth “red” matched in response “The car appears to be red”).
- **Soft Accuracy:** Partial word overlap score between prediction and ground truth.
- **Latency:** End-to-end inference time in milliseconds, measured on a single NVIDIA A100-SXM4-40GB GPU.

- **Memory Usage:** Peak GPU memory consumption in gigabytes.

5.5 Baseline

The baseline LLaVA-1.5 model achieves 57.5% containment accuracy, 58.75% soft accuracy, 921.5ms latency, and 13.16GB memory on VQAv2-small. All results are compared against this baseline.

5.6 FastV

FastV is built on the empirical observation that attention over visual tokens becomes increasingly sparse as depth increases in modern VLM decoders. In shallow layers, attention is broadly distributed across visual tokens, enabling spatial feature aggregation. However, in deeper layers, attention mass concentrates on a small set of anchor tokens - often system, instruction, or output tokens - while most visual tokens receive negligible attention.

This creates a pronounced imbalance: the majority of visual tokens continue to participate in expensive attention computation despite contributing little to the final prediction. FastV hypothesizes that, beyond early layers, many visual tokens are effectively redundant and can be safely removed without harming task performance. Consequently, FastV proposes pruning visual tokens after shallow layers, where sufficient information integration has already occurred.

FastV operates during the prefill stage of autoregressive inference and consists of the following steps:

- **Layer Selection:** A shallow pruning layer (K) is chosen (typically 1-3).
- **Token Scoring:** For each visual token, FastV computes the mean attention score received from all other tokens, averaging over heads.
- **Top-K Selection:** A fixed ratio (R) of visual tokens with the highest average attention scores is retained and all remaining visual tokens are dropped, reducing the key-value sequence length for subsequent layers.

This approach is entirely model-agnostic and requires no retraining, making it attractive as a plug-and-play inference optimization.

6 Experiments Results

Table 1 summarizes the performance of weight pruning methods and SparseVLM across sparsity levels from 5% to 30%.

6.1 Weight Pruning Results

Unstructured pruning maintains baseline accuracy (57.5%) across all sparsity levels, with a slight improvement to 60% at 20% sparsity. However, latency reduction is negligible (<3%) due to the lack of sparse matrix acceleration on standard GPUs.

Kernel and filter pruning cause catastrophic failure. Even at 5% sparsity, kernel pruning drops accuracy to 25% and filter pruning to 35%. At 30% sparsity, both methods achieve 0% accuracy. This demonstrates that CLIP’s pretrained representations are highly sensitive to structural modifications—each kernel and filter encodes essential visual-semantic knowledge that cannot be removed without fine-tuning.

6.2 SparseVLM Results

SparseVLM achieves stable accuracy (51–52%) across all sparsity levels with only a 5.5% drop from baseline. Critically, it provides consistent **18–20% latency reduction** (921ms → 740ms) by reducing visual tokens from 576 to 403 at 30% sparsity (Table 2). The higher memory usage (29.49GB vs 13.16GB) is due to attention score computation for token selection.

6.3 Token Pruning vs Weight Pruning

Our results demonstrate that **token reduction is more effective than weight reduction** for VLM efficiency. Weight pruning targets model parameters but does not reduce sequence length, providing minimal speedup without specialized hardware. In contrast, SparseVLM directly reduces the quadratic attention bottleneck by removing redundant visual tokens, achieving real speedup on standard GPUs while preserving model accuracy.

Table 1: Comparison of Pruning Methods on Vision-Language Model

Method	Sparsity (%)	Containment (%)	Soft Acc (%)	Latency (ms)	Memory (GB)
Baseline	0	57.5	58.75	921.5	13.16
Unstructured	5	57.5	58.75	907.9	13.16
	10	57.5	58.75	893.7	13.16
	15	57.5	58.75	890.5	13.16
	20	60.0	61.25	898.0	13.16
	25	57.5	58.75	901.8	13.16
	30	57.5	58.75	919.5	13.16
Kernel	5	25.0	25.00	627.7	13.16
	10	5.0	5.00	886.6	13.16
	15	2.5	2.50	942.1	13.16
	20	5.0	5.00	951.7	13.16
	25	2.5	2.50	936.7	13.16
	30	0.0	0.00	952.2	13.16
Filter	5	35.0	35.00	642.1	13.16
	10	10.0	10.00	737.1	13.16
	15	5.0	5.00	858.9	13.16
	20	2.5	2.50	944.8	13.16
	25	0.0	0.00	938.6	13.16
	30	0.0	0.00	946.3	13.16
SparseVLM	5	52.0	54.40	753.6	29.49
	10	52.0	54.40	755.8	29.49
	15	51.6	53.90	746.5	29.49
	20	51.8	54.00	745.4	29.49
	25	51.6	53.60	744.6	29.49
	30	51.0	53.20	739.6	29.49

Table 2: SparseVLM Token Reduction Analysis

Sparsity (%)	Tokens	Containment (%)	Latency (ms)
5	512	52.0	753.6
10	512	52.0	755.8
15	489	51.6	746.5
20	460	51.8	745.4
25	432	51.6	744.6
30	403	51.0	739.6

Our initial experiments confirmed that token pruning is fundamentally more effective than traditional weight-level pruning for Vision-Language Models (VLMs). While unstructured weight pruning preserves accuracy, it provides negligible inference speedup, and structured pruning methods (kernel and filter pruning) lead to catastrophic performance collapse due to the highly specialized and non-redundant nature of pretrained visual features. In contrast, vision token pruning directly targets the dominant computational bottleneck of VLMs - quadratic attention over visual tokens - yielding substantial efficiency gains with graceful accuracy degradation.

Table 3: Summary Comparison at 30% Sparsity

Method	Accuracy (%)	Latency (ms)	Speedup	Status
Baseline	57.5	921.5	–	Reference
Unstructured	57.5	919.5	0.2%	Safe, no speedup
Kernel	0.0	952.2	-3.3%	Failed
Filter	0.0	946.3	-2.7%	Failed
SparseVLM	51.0	739.6	19.7%	Best tradeoff

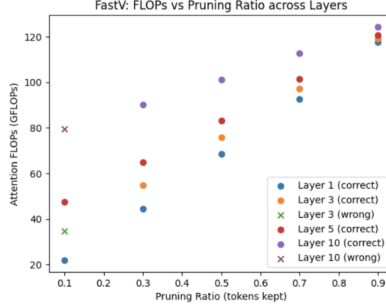


Figure 1: FastV.

6.4 FastV

Motivated by this observation, we turn to FastV Chen et al. [2024], a recent plug-and-play token pruning method that proposes an architecture-agnostic approach to accelerate VLM inference by removing redundant visual tokens after shallow layers.

We implemented FastV on PaLI-Gemma-3B-pt-224 and evaluated its performance on a visual question answering prompt (“What is this monument called?”). We tested pruning at layers (1, 3, 5, 10) and token retention ratios ranging from 0.9 to 0.1. (See tables 15, 14, 13, 12 and 11 for results).

The results show that pruning tokens immediately after shallow layers leads to substantial reductions in attention FLOPs, with prefill computation dropping from 130.4 GFLOPs (baseline) to as low as 34.7 GFLOPs at a 0.1 retention ratio. We estimate FLOPs by counting only the dominant attention operations:

$$\text{FLOPs} = 6BT_qd_{\text{model}}^2 + 4Bn_{\text{heads}}T_qT_kd_{\text{head}},$$

where B is the batch size, T_q the number of query tokens, T_k the number of key/value tokens, d_{model} the hidden dimension, n_{heads} the number of attention heads, and $d_{\text{head}} = d_{\text{model}}/n_{\text{heads}}$. This corresponds to linear projections and attention matrix multiplications. However, despite these efficiency gains, the generated answers were often underspecified or semantically degraded, frequently collapsing to generic responses such as “statue” or producing irrelevant outputs at aggressive pruning levels. (See image 1).

This reveals an important limitation of FastV: its token selection criterion - mean attention aggregation - is not robust enough to preserve task-critical visual information. Tokens that receive low average attention may still encode fine-grained, question-dependent semantics that are essential for precise answers.

6.5 SparseVLM + FastV

While FastV correctly identifies where pruning should occur, SparseVLM provides a more principled mechanism for deciding which tokens to prune. SparseVLM leverages text-guided cross-attention, explicitly modeling the relevance of each visual token to the input question. This allows it to retain visually salient regions conditioned on the linguistic query rather than relying on global attention statistics.

Table 4: Average performance of SparseVLM under different pruning strategies across 12 image-question pairs.

Pruned Layers	Token Keep Ratios	Avg FLOPs (GFLOPs)	Accuracy (%)
[1,2,3]	{1:0.9, 2:0.7, 3:0.5}	74.2	91.7
[1]	{1:0.3}	46.0	91.7
[1,2]	{1:0.5, 2:0.2}	36.9	91.7
[1]	{1:0.2}	34.9	83.3
[1]	{1:0.1}	23.4	83.3
[10]	{10:0.2}	86.6	66.7
[10]	{10:0.7}	114.1	75.0
[15,16,17]	{15:0.7, 16:0.5, 17:0.2}	120.5	83.3
[17]	{17:0.7}	129.9	83.3

Table 5: Effect of Visual Token Pruning at Different Layers

Method	Acc	Pruning in Early Layers (1,2)	Pruning in Later Layers (3–18)
Baseline Model	100%	NIL	NIL
Sparse VLM	90%	NIL	0.8
Sparse VLM	40%	NIL	0.9
Sparse VLM	30%	NIL	0.95
Sparse VLM	100%	0.2	NIL
Sparse VLM	90%	0.4	NIL
Sparse VLM	70%	0.6	NIL
Sparse VLM	30%	0.8	NIL

To combine the strengths of both approaches, we implement a hybrid pruning strategy: layer placement follows FastV’s insight, applying pruning at shallow or immediately post-shallow layers; token selection follows SparseVLM, retaining visual tokens that are most relevant to the input text. This design preserves FastV’s efficiency benefits while improving semantic fidelity through text-conditioned token importance.

We evaluated this combined strategy on the same PaLI-Gemma setup, measuring FLOPs and answer correctness across multiple pruning configurations (see table 4). The results demonstrate that:

- Pruning at shallow layers with SparseVLM selection consistently outperforms pure FastV, producing more specific and accurate answers (e.g., “statue of Christ the Redeemer”).
- Moderate pruning ratios (e.g., retaining 30–50% of tokens) achieve a strong trade-off, reducing prefill FLOPs by up to 65% while preserving semantic correctness.
- Pruning in deeper layers provides minimal computational benefit and often degrades accuracy, reinforcing the importance of early-layer pruning.

Notably, several configurations outperformed the unpruned baseline qualitatively, suggesting that removing visually irrelevant tokens can act as an implicit regularizer by suppressing noise.

6.5.1 Analysis

These results highlight a key insight: token pruning in VLMs is most effective when decoupling where pruning occurs from how tokens are selected. FastV correctly identifies shallow layers as the optimal pruning location but relies on a coarse attention heuristic. SparseVLM, by contrast, offers fine-grained, text-aware token selection but does not prescribe optimal pruning depth.

By combining the two, we obtain a pruning strategy that is: computationally efficient (FastV-style shallow pruning), semantically robust (SparseVLM-style token importance), fully training-free and inference-time only.

This hybrid approach demonstrates that text-guided token pruning at shallow layers can outperform both raw PaLI-Gemma and naive FastV pruning, producing better answers at significantly lower computational cost.

Table 6: Visual Token Pruning in Decoder Layers

Method	Acc	Pruning
Baseline Model	100%	NIL
Sparse VLM	100%	0.2
Sparse VLM	90%	0.4
Sparse VLM	70%	0.6
Sparse VLM	30%	0.8

Table 7: Simple Visual Token Pruning in Encoder

Method	Acc	Pruning
Baseline Model	100%	NIL
Simple VT Pruning in Encoder	90%	0.2
Simple VT Pruning in Encoder	50%	0.4
Simple VT Pruning in Encoder	50%	0.6
Simple VT Pruning in Encoder	30%	0.8

7 Conclusion

We present a systematic study of sparsity strategies for Vision–Language Models (VLMs), showing that vision token pruning is more effective than weight-level pruning for inference acceleration. Across pruning choices (what, where, and how to prune) token-level sparsification aligns better with the computational structure of modern VLMs.

We find that weight pruning offers limited benefits: unstructured pruning yields negligible speedups, while structured methods often severely degrade performance due to the specialized and non-redundant nature of pretrained visual features. In contrast, token pruning directly targets the dominant bottleneck - quadratic attention over visual tokens - making it a more effective efficiency mechanism.

We further show that decoder-side, text-guided token pruning consistently outperforms encoder-side approaches, particularly at higher sparsity levels. Comparing SparseVLM and FastV, SparseVLM provides robust text-aware token selection, while FastV highlights the importance of pruning at shallow layers but relies on a coarse heuristic that can discard semantically important regions.

Combining these insights, we integrate FastV-style shallow-layer pruning with SparseVLM’s text-guided selection, achieving superior accuracy–efficiency trade-offs. This hybrid approach consistently outperforms either method alone and, in some cases, even surpasses the unpruned baseline, indicating that well-designed token pruning can also suppress visual noise.

Overall, our findings lead to three key takeaways. First, token pruning is strictly more effective than weight pruning for VLM inference acceleration. Second, pruning location and token selection should be treated as separate design decisions, as they capture orthogonal aspects of the problem. Third, text-guided token selection applied at shallow or early decoder layers offers the best balance between efficiency and accuracy.

We believe this work clarifies the practical and conceptual trade-offs in VLM sparsification and provides a principled foundation for future research. Promising directions include pruning VLMs on the head dimension and introducing gaze features to better select tokens to prune. Taken together, our results suggest that token-level sparsification is a central tool for scaling multimodal models efficiently without sacrificing semantic fidelity.

References

- Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models, 2024.
- Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. Improved baselines with visual instruction tuning, 2024. URL <https://arxiv.org/abs/2310.03744>.

Table 8: Group-wise Visual Token Pruning in Encoders (IVTP, ECCV 2024)

Method	Acc	Pruning
Baseline Model	100%	NIL
Group-wise Token Pruning	100%	0.2
Group-wise Token Pruning	70%	0.4
Group-wise Token Pruning	60%	0.6
Group-wise Token Pruning	50%	0.8

Table 9: Visual Token Pruning in Both Encoder (Simple Pruning) and Decoder (Text-Guided)

Method	Acc	Encoder Pruning	Decoder Pruning
Baseline Model	100%	NIL	NIL
Method	90%	0.2	0.2
Method	80%	0.2	0.4
Method	70%	0.2	0.6
Method	50%	0.2	0.8
Method	50%	0.4	0.2
Method	50%	0.4	0.4
Method	50%	0.4	0.6
Method	40%	0.4	0.8

Yuan Zhang, Chun-Kai Fan, Junpeng Ma, Wenzhao Zheng, Tao Huang, Kuan Cheng, Denis Gudovskiy, Tomoyuki Okuno, Yohei Nakata, Kurt Keutzer, and Shanghang Zhang. Sparse-
 vlm: Visual token sparsification for efficient vision-language model inference, 2025. URL
<https://arxiv.org/abs/2410.04417>.

A Appendix / supplemental material

Table 10: Visual Token Pruning in Both Encoder (Group-wise) and Decoder (Text-Guided)

Method	Acc	Encoder Pruning	Decoder Pruning
Baseline Model	100%	NIL	NIL
Method	100%	0.2	0.2
Method	80%	0.2	0.4
Method	80%	0.2	0.6
Method	50%	0.2	0.8
Method	70%	0.4	0.2
Method	70%	0.4	0.4
Method	70%	0.4	0.6
Method	70%	0.4	0.8

Pruning VT in Decoders vs Encoders

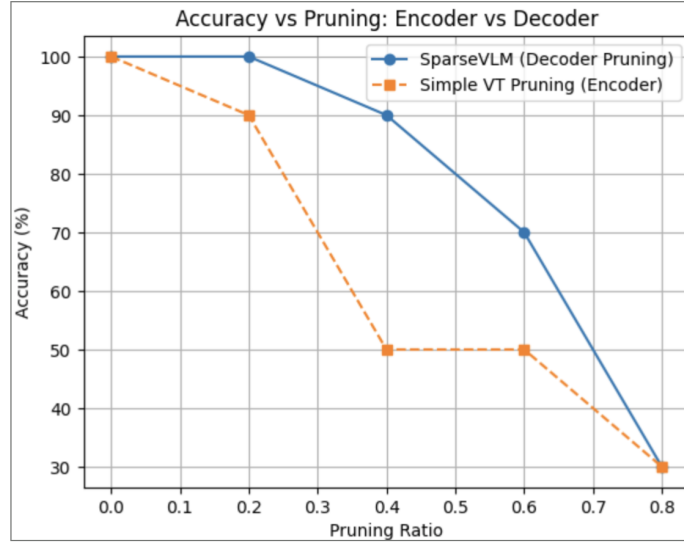


Figure 2: Pruning VT in Decoders vs Encoders

Group-wise VT Pruning vs Simple Pruning VT in Encoders

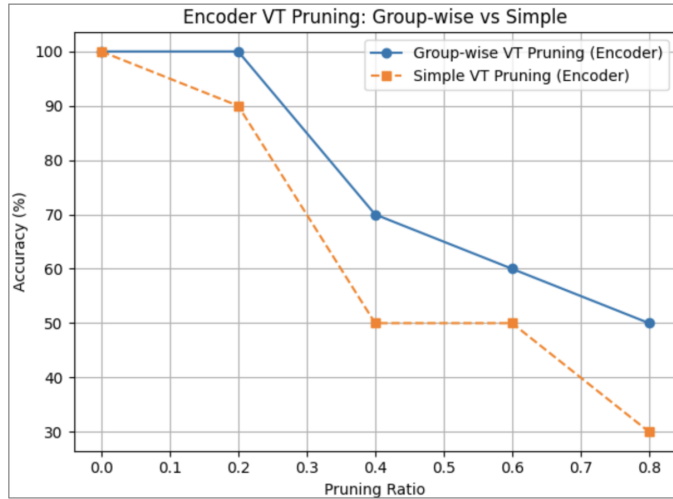


Figure 3: Group-wise Token Pruning VT in Encoders Method from paper

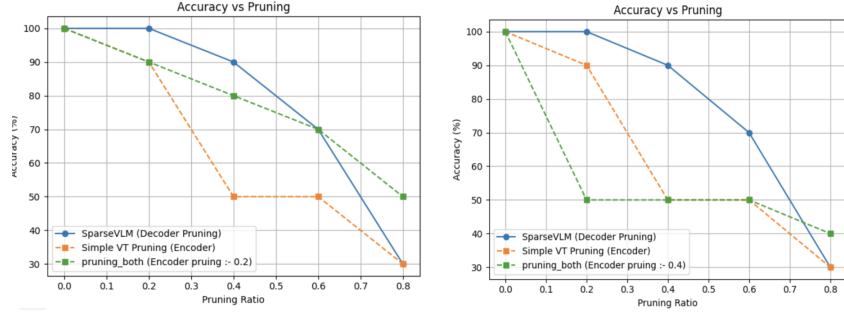


Figure 4: Pruning VT in both Encoder(Simple pruning) and Decoder(Text -Guided) Vs Pruning VT in Decoders Vs Simple Pruning VT in Encoders

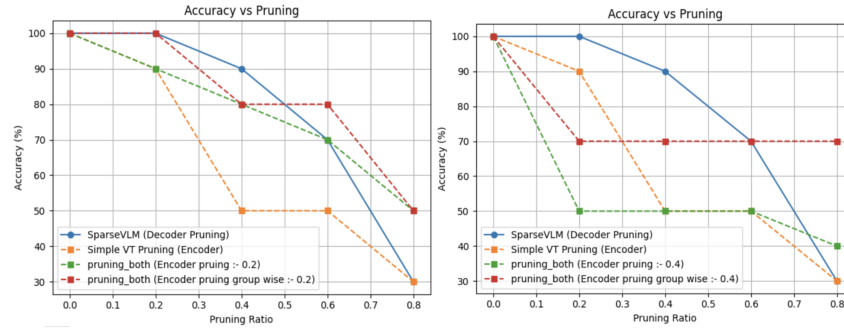


Figure 5: Pruning VT in both Encoder(Simple pruning) and Decoder(Text -Guided) Vs Pruning VT in Decoders Vs Simple Pruning VT in Encoders

Table 11: Raw PaliGemma baseline result on the Christ the Redeemer image.

Method	Pruning	Output	Correct	Total Attn FLOPs (GFLOPs)
Raw PaliGemma	None	statue	✓	130.396

Table 12: FastV results on Christ the Redeemer image with pruning at Layer 10. Correct answer is “statue”.

Layer	Token Ratio	Output	Total FLOPs (GFLOPs)
10	0.9	statue	124.303
10	0.7	statue	112.608
10	0.5	statue	101.254
10	0.3	statue	90.029
10	0.1	most beautiful places in the world	79.363

Table 13: FastV results on Christ the Redeemer image with pruning at Layer 5.

Layer	Token Ratio	Output	Total FLOPs (GFLOPs)
5	0.9	statue	120.495
5	0.7	statue	101.490
5	0.5	statue	83.040
5	0.3	statue	64.799
5	0.1	statue	47.468

Table 14: FastV results on Christ the Redeemer image with pruning at Layer 3.

Layer	Token Ratio	Output	Total FLOPs (GFLOPs)
3	0.9	statue	118.971
3	0.7	statue	97.043
3	0.5	statue	75.755
3	0.3	statue	54.707
3	0.1	?	34.709

Table 15: FastV results on Christ the Redeemer image with pruning at Layer 1.

Layer	Token Ratio	Output	Total FLOPs (GFLOPs)
1	0.9	statue	117.448
1	0.7	statue	92.596
1	0.5	statue	68.469
1	0.3	statue	44.615
1	0.1	statue	21.951

Table 16: Effect of pruning layer and ratio on semantic correctness for the Christ the Redeemer image.

ID	Pruned Layers	Token Keep Ratios	FLOPs (GFLOPs)	Output Quality
A	[1,2,3]	{1:0.9, 2:0.7, 3:0.5}	72.77	Correct
B	[1]	{1:0.3}	44.62	Correct
D	[1]	{1:0.2}	33.42	Incorrect
C	[1]	{1:0.1}	21.95	Incorrect
E	[1,2]	{1:0.5, 2:0.2}	35.48	Correct
J	[3]	{3:0.1}	34.71	Incorrect
F	[10]	{10:0.2}	84.76	Partial
G	[10]	{10:0.7}	112.61	Incorrect
H	[17]	{17:0.7}	128.17	Partial
I	[15,16,17]	{15:0.7, 16:0.5, 17:0.2}	118.83	Partial